

Raport Temă Practică ML

Aldea Alexandra-Maria, Cotea Carla-Gabriela

Ianuarie 2026

1 Preprocesare (Task 2.1)

1.1 Clasificarea Produselor

Scop

Primul pas a fost împărțirea produselor pe categorii generalizate din produsele particulare din datasetul original (de exemplu, “Pepsi”, “Crazy Schnitzel”, “French fries”). De asemenea, s-au adăugat coloanele de `tip_zi` (weekday/weekend) și `perioada_zi` (morning/lunch/evening).

Implementare

Scriptul `add_category_column.py` definește o funcție care primește numele produsului și, pe baza unor liste predefinite, atribuie o categorie.

```
def categorize_product(product_name):  
    if 'Pepsi' in product_name or 'Aqua' in product_name:  
        return 'drink'  
    elif product_name in ['Crazy_Sauce', 'Cheddar_Sauce', ...]:  
        return 'sauce'  
    elif 'Schnitzel' in product_name:  
        return 'schnitzel'  
    # ... etc
```

Categorii Definite

Am împărțit toate produsele în 9 categorii semantice:

- **drink**: Băuturi (Pepsi, Aqua, Lipton, etc.)
- **sauce**: Sosuri comandate standalone (Crazy Sauce, Cheddar Sauce, etc.)
- **schnitzel**: Schnitzele
- **side_dish**: Garnituri (cartofi)
- **main_dish**: Fel principal (Mac & Cheese, Chicken Bao Buns)
- **extra**: Ingrediente extra
- **salad**: Salate
- **dessert**: Deserturi
- **packaging**: Ambalaj

Exemplu după categorizare

Tabela 1: Exemplu date după categorizare

id_bon	Product Name	Category	tip_zi	perioada_zi
1	Crazy Schnitzel	schnitzel	weekday	lunch
1	Crazy Sauce	sauce	weekday	lunch
1	Pepsi	drink	weekday	lunch
2	Crazy Schnitzel	schnitzel	weekend	evening
2	French fries	side_dish	weekend	evening

1.2 Agregare la Nivel de Bon

Problema

După pasul anterior, datele sunt la nivel de produs individual. Pentru a antrena un model de clasificare la nivel de bon, trebuie agregate informațiile tuturor produselor pe fiecare bon.

Transformarea Datelor

Scriptul `create_modified_dataset.py` transformă datele prin:

1. Grupare după `id_bon`.
2. Pentru fiecare bon, numărare a produselor per categorie.
3. Pentru fiecare bon, numărare a fiecărui produs specific.
4. Creare de coloane noi: `count_category_*` și `count_product_*`.

Exemplu Transformare

ÎNAINTE (nivel de produs):

id_bon	retail_product_name	product_category
1	Crazy Schnitzel	schnitzel
1	Crazy Sauce	sauce
1	Pepsi	drink
2	Crazy Schnitzel	schnitzel
2	French fries	side_dish

DUPĂ (nivel de bon):

id_bon	count_cat_schnitzel	count_cat_sauce	count_prod_sauce	...
1	1	1	1	
2	1	0	0	

1.3 Filtrare și Extragere Features Finale

Context

Scriptul `filter_crazy_schnitzel+features.py` realizează următoarele:

1. Filtrează doar bonuri care conțin *Crazy Schnitzel*.
2. Creează eticheta target.
3. Computează caracteristici de coș.

1. Filtrare Inițială

```
bonuri_cu_crazy_schnitzel = df[df['retail_product_name'] == 'Crazy_Schnitzel']['id_bon'].unique()
df = df[df['id_bon'].isin(bonuri_cu_crazy_schnitzel)]
```

2. Eticheta Target (y)

```
bonuri_cu_crazy_sauce = df[df['retail_product_name'] == 'Crazy_Sauce']['id_bon'].unique()
df['y'] = df['id_bon'].apply(lambda x: 1 if x in bonuri_cu_crazy_sauce else 0)
```

Variabila target este binară:

- $y = 1$: Bonul conține *Crazy Sauce*.
- $y = 0$: Bonul NU conține *Crazy Sauce*.

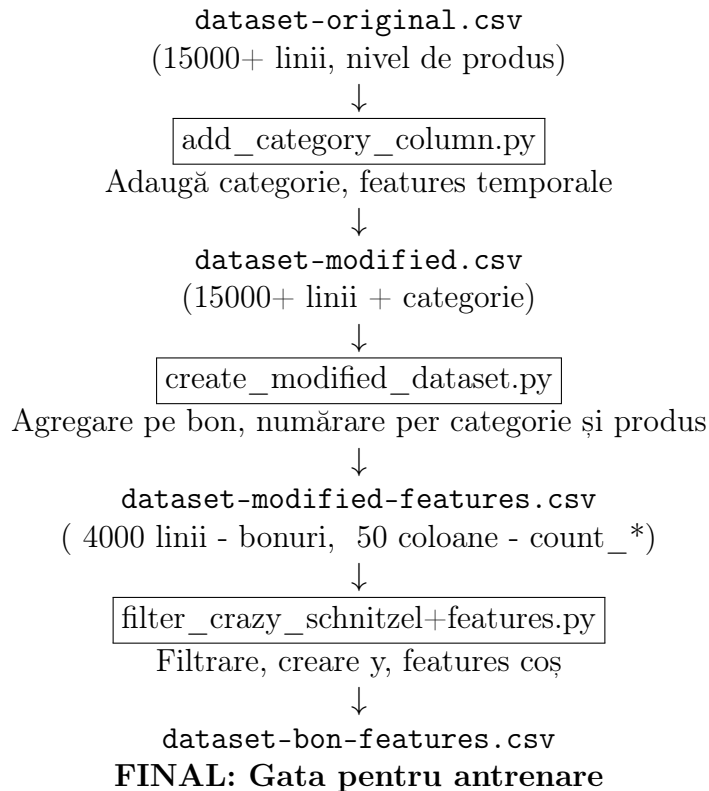
3. Caracteristici de Coș

Pentru fiecare bon, calculez caracteristici agregate:

Tabela 2: Caracteristici de coș

Feature	Descriere
<code>cart_size</code>	Numărul total de produse pe bon
<code>distinct_products</code>	Numărul de produse DIFERITE comandate
<code>total_value</code>	Valoarea totală a comenzii (în bani)
<code>avg_price</code>	Preț mediu per produs
<code>count_category_*</code>	Numărul de produse din fiecare categorie

Rezumat Complet al Fluxului



2 Preprocesare (Task 2.2)

Același flux de preprocesare a fost aplicat și pentru problema 2.2. Diferența majoră a constatat în absența etapei de filtrare, utilizându-se toate bonurile disponibile, nu doar cele care conțineau produsul *Crazy Schnitzel*. Variabila țintă (y) a fost, pe rând, fiecare dintre cele 8 sosuri posibile, la antrenare acesta fiind exclus din trăsăturile furnizate pentru a evita “scurgerile de date” (*data leakage*).

3 Regresie Logistică (Task 2.1)

Această secțiune detaliază procesul iterativ prin care am optimizat implementarea proprie a Regresiei Logistice, deciziile luate privind preprocesarea datelor și selecția hiperparametrilor, precum și comparația finală cu implementarea de referință din `scikit-learn`.

3.1 Implementare finală

Pentru rezolvarea problemei de clasificare binară (predicția sosului *Crazy Sauce*), a fost implementată Regresia Logistică folosind metoda Gradient Descent. Minimizarea erorii s-a realizat folosind funcția de cost Binary Cross-Entropy, aceasta fiind standardul pentru problemele de clasificare binară.

În varianta finală, am implementat mecanismul de regularizare L2 pentru a penaliza ponderile excesive și a evita overfitting-ul. A fost inclus și un mecanism de early stopping setat să se declanșeze după 5 epoci consecutive în care funcția de loss crește, deși acesta nu a fost declanșat pe parcursul experimentelor finale, modelul convergând stabil.

Un aspect crucial pentru eficiența algoritmului de optimizare a fost preprocesarea datelor prin standardizare. Am transformat variabilele de intrare astfel încât să aibă media 0 și deviația

standard 1. Această uniformizare a suprafeței de eroare a permis utilizarea unei rate de învățare agresive ($learning_rate = 0.5$) fără riscul divergenței.

Nu în ultimul rând, stabilitatea numerică la începutul antrenării a fost asigurată prin inițializarea ponderilor folosind metoda Xavier, prevenind astfel saturația prematură a funcției de activare Sigmoid.

3.2 Evoluția experimentului

Performanța modelului a fost obținută printr-un proces iterativ de optimizare, rezultatele fiind monitorizate constant comparativ cu baseline-ul `sklearn`.

În prima fază a experimentelor, am rulat modelul pe datele brute cu regularizare L2 și un $lr = 0.1$. Această abordare a rezultat într-o acuratețe modestă de aproximativ 65.83%, sub-performând masiv față de implementarea de referință (76.47%). Diferența de peste 10% a indicat clar că datele necesită o scalare corespunzătoare.

Un salt major a fost realizat prin adăugarea de trăsături de tip contor (*counter features*). Maparea produselor specifice și generarea unor contoare per categorie au permis modelului să generalizeze comportamentul clienților. Această modificare a propulsat acuratețea la aproximativ 87% , după normalizarea datelor ajungând ulterior la 96.64%.

Ulterior, am setat o rată de învățare agresivă ($learning_rate = 0.25$) combinată cu mecanismul de Early Stopping ($patience = 5$ epoci), însă, în mod surprinzător, acesta nu a fost declanșat pe parcursul experimentelor.

Diferența critică a fost făcută de numărul de iterații. Rularea pentru 10.000 de iterații a adus modelul la paritate cu `sklearn` (99.16%), iar extinderea la 15.000 de iterații a permis depășirea baseline-ului pe metricile punctuale, atingând o acuratețe de 99.44%.

Am mai experimentat și testând limitele absolute ale implementării prin rularea a 50.000 de iterații, am reușit să minimizez funcția de loss până la valoarea de 0.0160, și se putea observa de pe plot-ul funcției de loss că aceasta încă scădea.

Este important de menționat și un experiment care nu a dat rezultatele așteptate: implementarea unui *Learning Rate Decay*. Scăderea treptată a ratei de învățare a degradat performanța la nivelul etapelor intermediare ($Acc \approx 98.32\%$), deoarece rata scădea prea rapid, împiedicând modelul să parcurgă pașii necesari pentru a ajunge în zona de minim global atinsă anterior cu rata constantă.

3.3 Evaluarea Modelului și Interpretarea Rezultatelor

Tabela 3: Performanța Finală: Implementare Proprie vs. Scikit-Learn

Metrică	Custom Implementation	sklearn Baseline	Diferență
Accuracy	0.9944	0.9916	+0.0028%
Precision	0.9896	0.9895	+0.0001%
Recall	1.0000	0.9947	+0.0053%
F1-Score	0.9948	0.9921	+0.0026%
ROC-AUC	0.9967	0.9995	-0.0028%

3.4 Matricea de Confuzie

Pentru a vizualiza performanța modelului pe setul de testare, am generat matricea de confuzie (Figura 1). Aceasta ilustrează capacitatea modelului de a distinge corect între bonurile care conțin *Crazy Sauce* și cele care nu.

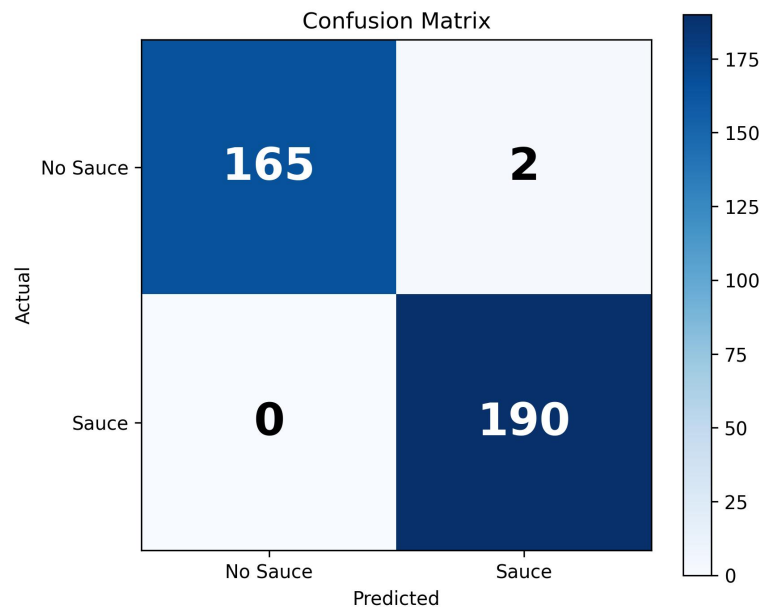


Figura 1: Matricea de Confuzie pentru clasificarea binară a sosului Crazy Sauce.

Rezultatele indică o precizie extrem de ridicată:

- **True Negatives (TN):** 165 (Bonuri fără sos clasificate corect)
- **True Positives (TP):** 190 (Bonuri cu sos clasificate corect)
- **False Positives (FP):** 2 (Bonuri fără sos clasificate greșit ca având sos)
- **False Negatives (FN):** 0 (Niciun bon cu sos nu a fost ratat)

3.5 Analiza Curbei ROC

Graficul demonstrează o capacitate de discriminare aproape ideală pentru ambele modele, curbele tinzând rapid către colțul stânga-sus (Rata de Pozitive Adevărate = 1, Rata de Pozitive False = 0). praguri de discriminare (Figura 2).

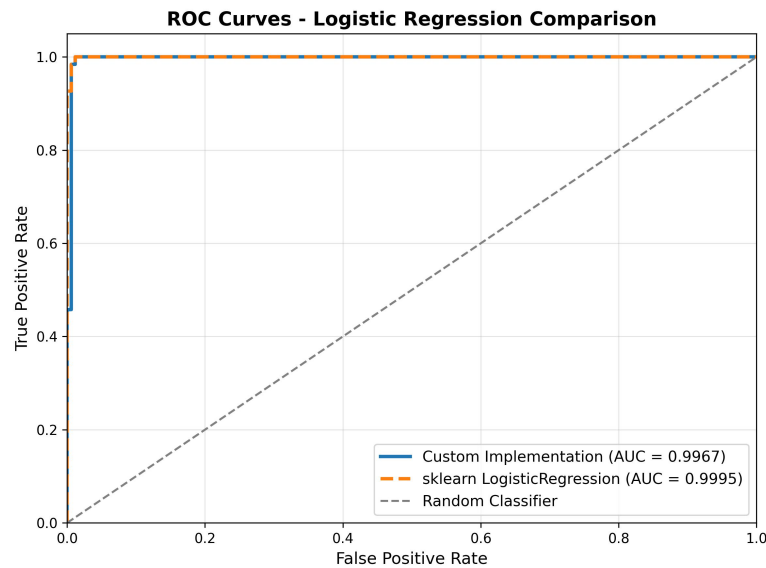


Figura 2: Comparație curbe ROC: Implementare Proprie vs. Scikit-Learn.

3.6 Interpretarea Coeficienților Regresiei Logistice

Un avantaj major al Regresiei Logistice este interpretabilitatea. Analizând ponderile (w) asociate fiecărei trăsături, putem înțelege ce factori influențează decizia clientului de a cumpăra *Crazy Sauce*.

Factori care CRESC probabilitatea de cumpărare (Coeficienți Pozitivi)

- **Mărimea Coșului (`cart_size`, +12.04):** Acesta este de departe cel mai puternic predictor. Cu cât un client comandă mai multe produse, cu atât este mai probabil să comande și un sos.
- **Valoarea Totală (`total_value`, +3.97):** Corelat cu mărimea coșului, clienții care cheltuie mai mult sunt mai dispuși să adauge produse complementare.
- **Diversitatea (`distinct_products`, +1.98):** O varietate mai mare de produse în coș indică o masă complexă, care necesită adesea sosuri.

Factori care SCAD probabilitatea de cumpărare (Coeficienți Negativi)

- **Alte Sosuri (`has_cheddar_sauce`, -4.91; `has_garlic_sauce`, -4.19):** Dacă un client a comandat deja un alt tip de sos, probabilitatea de a comanda și *Crazy Sauce* scade drastic. Aceasta confirmă intuiția că produsele din aceeași categorie sunt substituibile.
- **Prețul Mediu (`avg_price`, -3.10):** Un preț mediu ridicat ar putea indica comanda unor produse scumpe care nu necesită sos (de exemplu, salate complexe sau meniuri care includ deja sosuri).
- **Băuturile (`count_drink`, -1.92):** Clienții care comandă preponderent băuturi (probabil doar o gustare sau o pauză de cafea) sunt mai puțin interesați de sosuri.

4 Sistem de Recomandare Sosuri (Task 2.2)

Extinzând arhitectura validată la punctul anterior, am dezvoltat un sistem de recomandare capabil să sugereze sosul ideal în funcție de componența coșului de cumpărături.

4.1 Abordare și Implementare

Deoarece problema implică recomandarea unuia dintre cele 8 sosuri posibile, am adoptat strategia One-vs-All. Arhitectura de regresie logistică utilizată este identică cu cea de la 2.1, însă aplicată pe setul extins de date.

Diferențe față de Task 2.1

Spre deosebire de abordarea anterioară, implementarea pentru sistemul de recomandare a necesitat câteva ajustări, majoritatea la nivel de preprocesare. A fost folosit datasetul extins (nu doar bonurile ce conțin Crazy Schnitzel).

Arhitectura a fost adaptată prin antrenarea a 8 modele distincte, câte unul pentru fiecare tip de sos (*Crazy Sauce*, *Cheddar*, *Garlic*, etc.), trecând astfel la o strategie de tip One-vs-All. Înainte de antrenarea fiecărui sos a fost eliminată coloana caracteristică sosului curent pentru a nu avea scurgeri de date.

4.2 Rezultate Experimentale

Performanța la Nivel de Clasificator Individual

Fiecare dintre cele 8 modele a converș după 15000 de iterații ($\max(\text{Loss}) \approx 0.0024$).

Tabela 4: Acuratețea modelelor individuale per tip de sos

Tip Sos	Final Loss	Accuracy
Crazy Sauce	0.0022	0.9994
Cheddar Sauce	0.0024	1.0000
Extra Cheddar Sauce	0.0007	1.0000
Garlic Sauce	0.0023	0.9987
Tomato Sauce	0.0024	0.9994
Blueberry Sauce	0.0024	1.0000
Spicy Sauce	0.0024	1.0000
Pink Sauce	0.0023	0.9987

Performanța Sistemului de Recomandare

Pentru evaluare, am comparat sistemul nostru cu un **Baseline de Popularitate**. Metrica principală, **Hit@K**, măsoară dacă sosul real cumpărat de client se află în primele K sugestii ale modelului.

4.3 Interpretarea Rezultatelor

Rezultatele obținute sunt categorice:

Tabela 5: Comparație Model Recomandare vs. Baseline Popularitate globală

Metrică	K	Model Propriu	Baseline (Popularitate)	Improvement
Hit@1	1	1.0000	0.7975	+20.25%
Hit@3	3	1.0000	0.7975	+20.25%
Hit@5	5	1.0000	1.0000	+0.00%
Precision@1	1	1.0000	0.7975	+20.25%
Precision@3	3	0.3333	0.2658	+6.75%
Precision@5	5	0.2000	0.2000	+0.00%

- **Hit@1 (1.0000):** Modelul a reușit să prezică corect sosul ales de client *de fiecare dată* pe prima poziție. Aceasta indică faptul că, în datele analizate, alegerea sosului este puternic determinată în funcție de produsele din coș (ex: *Blueberry Sauce* este probabil asociat exclusiv cu un desert specific sau un produs de tip "Papanasi").
- **Superioritate față de Baseline:** Deși popularitatea generală oferă o ghicire bună (79.75%), modelul acoperă "golul" de 20%, personalizând perfect recomandarea pentru cazurile mai rare sau specifice.

5 Sistem de Recomandare Upsell prin Ranking (Task 3)

5.1 Descrierea problemei

Pentru acest Task, a trebuit să facem un sistem care generează un **ranking de produse pentru upsell**, pornind de la un coș parțial de cumpărături și momentul și tipul zilei. Prin upsell înțelegem recomandarea unui produs suplimentar cu probabilitate mare de a fi cumpărat și cu impact pozitiv asupra venitului.

Problema este formulată ca o problemă de **ranking**: pentru un coș parțial, algoritmul trebuie să ordoneze produsele candidate astfel încât produsul lipsă (eliminat artificial) să apară cât mai sus în listă.

Formal, pentru un produs p și un coș C , se urmărește maximizarea unui scor de forma:

$$Score(p | C) = P(p | C, context) \cdot f(p)$$

unde $f(p)$ ține cont de preț și popularitate.

5.2 Descrierea dataset-ului

Dataset-ul conține tranzacții (bonuri fiscale) din retail, fiecare rând reprezentând un produs vândut. Câmpurile relevante utilizate sunt:

- `id_bon` – identificatorul bonului
- `retail_product_name` – numele produsului
- `SalePriceWithVAT` – prețul produsului
- `data_bon` – data și ora tranzacției

Un bon conține mai multe produse.

5.3 Preprocesare

Pașii de preprocesare au fost următorii:

1. Conversia câmpului `data_bon` la format `datetime`.
2. Extracția a două variabile de context:
 - **tip_zi**: weekday / weekend
 - **perioada**: dimineață, prânz, seară, noapte
3. Filtrarea produselor doar la o categorie de interes pentru upsell:
 - băuturi
 - sosuri
 - garnituri

Filtrarea s-a făcut folosind cuvinte-cheie în numele produselor.

4. Eliminarea bonurilor cu mai puțin de două produse.
5. Împărțirea dataset-ului în **train (80%)** și **test (20%)** la nivel de bon.

5.4 Construcția setului de antrenare

Setul de antrenare a fost construit pornind de la bonurile complete din dataset. Pentru fiecare bon, s-au generat mai multe exemple prin eliminarea, pe rând, a câte unui produs din coș. Produsele rămase formează coșul de intrare, iar produsul eliminat este considerat produsul țintă care trebuie recomandat de algoritm.

Această abordare simulează un scenariu realist: clientul a adăugat deja anumite produse în coș, iar sistemul încearcă să recomande un produs suplimentar relevant. În același timp, metoda permite evaluarea obiectivă a performanței, deoarece produsul eliminat este cunoscut și poate fi verificat dacă apare în topul recomandărilor.

5.5 Algoritmi utilizați

Pentru rezolvarea problemei de ranking pentru upsell au fost implementați mai mulți algoritmi, atât modele simple de tip baseline, cât și metode care folosesc informația din coș și din context. Scopul a fost compararea performanței acestora în același cadru experimental.

5.5.1 Baseline-uri

Ca punct de referință, au fost utilizate două baseline-uri. Primul baseline este bazat pe popularitate și ordonează produsele descrescător după numărul total de apariții în setul de antrenare. Al doilea baseline este bazat pe venit și ordonează produsele după suma totală a veniturilor generate de fiecare produs.

Aceste metode nu folosesc informația din coșul curent sau contextul temporal, însă oferă o estimare simplă a performanței care poate fi obținută fără un model de învățare automată și le folosim pentru a evalua dacă modelele mai complexe aduc un câștig real.

5.5.2 Naive Bayes

Modelul Naive Bayes este implementat în clasa `NaiveBayesUpsell`. Antrenarea se face prin funcția `fit`, unde se calculează probabilitatea fiecărui produs de a apărea ca upsell, pe baza frecvenței sale în datele de antrenare. Aceste valori sunt salvate în `self.prior`. În aceeași funcție se construiesc și probabilitățile condiționate pentru produsele din coș și pentru context, stocate în `self.likelihood`.

Produsele din coș sunt tratate ca variabile binare, iar contextul (tipul zilei și perioada zilei) este introdus separat, cu o influență mai mică. Pentru a evita situațiile în care o combinație nu apare deloc în date, am folosit smoothing direct în implementare. Predicția este realizată în funcția `predict_proba`, iar la final scorurile sunt normalizate.

Principalul avantaj al acestui model este că este simplu, rapid și ușor de controlat, dar dezavantajul este ipoteza de independență dintre produse, care nu este mereu realistă, mai ales când anumite produse sunt cumpărate frecvent împreună.

5.5.3 k-NN pe vectori binari

Pentru metoda k-NN, coșurile sunt transformate în vectori binari folosind `MultiLabelBinarizer` pentru produse și `OneHotEncoder` pentru context. Acești vectori sunt construiți înainte de antrenare și folosiți direct în clasa `KNNUpsellVector`. În funcția `fit` sunt salvate datele de antrenare, fără a exista un proces de antrenare propriu-zis.

Predicția se face în funcția `predict_proba`, unde se calculează similaritatea dintre coșul curent și toate coșurile din setul de antrenare. Similaritatea este de tip Jaccard și se bazează pe intersecția și reuniunea vectorilor binari. După asta, sunt aleși cei mai apropiați `k` vecini, iar produsele primesc un scor, în funcție de cât de similare sunt coșurile respective.

Avantajul acestui model este că poate surprinde relații locale între produse fără a face presupuneri teoretice complicate, dar dezavantajul este costul computațional la predicție și faptul că performanța depinde destul de mult de alegerea parametrului `k`.

5.5.4 ID3

Algoritmul ID3 este implementat prin clasa `ID3Node`, care construiește recursiv un arbore de decizie. Funcția `fit` este responsabilă de antrenarea arborelui și, la fiecare nivel, alege produsul care aduce cel mai mare Information Gain, calculat folosind funcția `entropy`. Split-urile sunt făcute verificând dacă un produs este sau nu prezent în coș.

Pentru a evita ca arborele să devină prea complex, adâncimea maximă este limitată prin parametrul `max_depth`. În nodurile finale nu se salvează o singură clasă, ci o distribuție de probabilitate peste produsele posibile, ceea ce permite folosirea funcției `predict_proba` pentru ranking.

Un mare avantaj al acestui model este interpretabilitatea, deoarece deciziile pot fi urmărite ușor. Dezavantajul este, totuși, că un singur arbore nu generalizează foarte bine și poate fi influențat de zgometul din date, ceea ce îi limitează performanța.

5.5.5 AdaBoost

Modelul AdaBoost este implementat în clasa `AdaBoostUpsell` și folosește mai mulți arbori ID3 de adâncime mică drept clasificatori slabi. În funcția `fit`, fiecare arbore este antrenat pe aceleași date, dar cu ponderi diferite pentru exemple, astfel încât exemplele greșite anterior să conteze mai mult la pașii următori.

Pentru fiecare arbore se calculează un coeficient de importanță, iar predicția finală este obținută în funcția `predict_proba`, unde distribuțiile de probabilitate returnate de fiecare arbore sunt combinate într-un scor final.

Avantajul principal la AdaBoost este că oferă rezultate mai bune și mai stabile decât un singur arbore de decizie. Pe de altă parte, antrenarea durează mai mult și poate fi sensibil la date zgomotoase.

5.6 Funcția de ranking

Ranking-ul final este construit folosind funcția `rank_from_proba`. Aceasta primește probabilitățile estimate de un model și coșul curent, apoi selectează un set de produse candidate cu ajutorul funcției `generate_candidates`.

Pentru fiecare produs candidat, scorul final este calculat combinând probabilitatea estimată cu informații despre preț și popularitate, folosind dicționarele `prices` și `popularity`. În acest fel, sistemul încearcă să recomande produse care sunt atât relevante pentru client, cât și avantajoase din punct de vedere economic, folosind formula:

$$Score(p) = P(p)^{0.85} \cdot price(p)^{0.1} \cdot popularity(p)^{0.05}$$

5.7 Metodologia de evaluare

Evaluarea a fost realizată folosind setul de test, construit pe baza coșurilor parțiale. Pentru fiecare exemplu, algoritmul generează un ranking de produse candidate, iar performanța este măsurată verificând dacă produsul eliminat apare în primele poziții ale ranking-ului.

Au fost utilizate metricile Hit@1, Hit@3 și Hit@5, care măsoară proporția de exemple pentru care produsul corect apare în primele K poziții.

5.8 Rezultate

Tabela 6: Performanță Ranking Upsell (Hit@K)

Model	Hit@1	Hit@3	Hit@5
Popularity Baseline	0.184	0.463	0.655
Revenue Baseline	0.184	0.415	0.597
Naive Bayes	0.348	0.727	0.834
KNN	0.316	0.679	0.785
ID3	0.336	0.711	0.836
AdaBoost	0.323	0.682	0.813

5.8.1 Variante încercate și observații negative

Am încercat și analizat mai multe variante care nu au condus la îmbunătățiri semnificative ale performanței. De exemplu, realizarea ranking-ului peste întregul set de produse, fără o etapă de generare a candidaților, a dus la rezultate mai slabe și la creșterea zgomotului în recomandări.

De asemenea, utilizarea unui arbore ID3 fără limitarea adâncimii a condus la supraînvățare (overfitting), cu performanță scăzută pe setul de test. În cazul metodei k-NN, valori prea mari ale parametrului k au diluat similaritatea locală dintre coșuri.

Bazându-ne pe aceste rezultate, am făcut alegerile finale pentru acest task.

5.8.2 Evoluția performanței în funcție de K

Acest grafic evidențiază comportamentul modelelor în funcție de dimensiunea listei de recomandări. Se observă că modelele care folosesc informația din coș cresc mai rapid și ating valori ridicate de Hit@5, ceea ce le face potrivite pentru un sistem real de upsell, unde nu este necesar ca produsul corect să fie mereu pe prima poziție

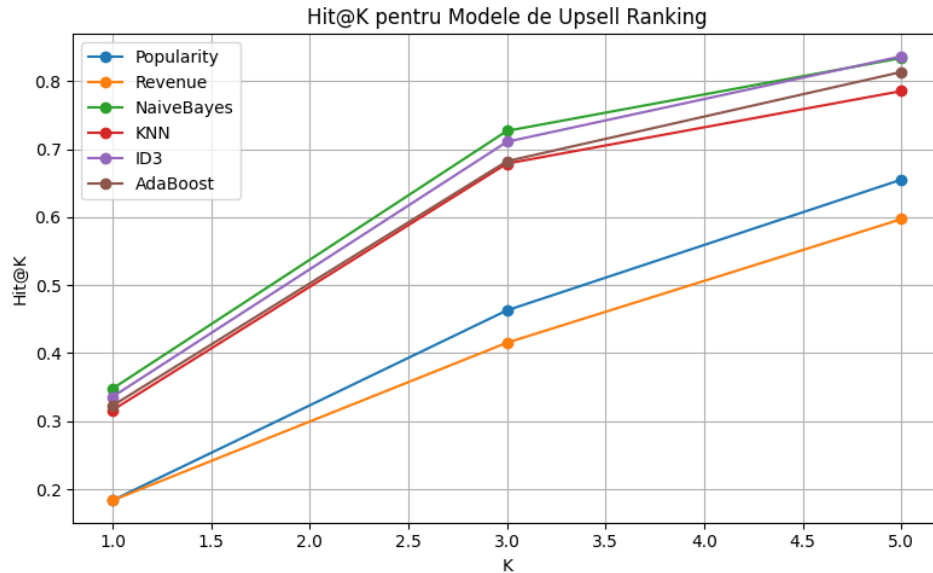


Figura 3: Evoluția metricii Hit@K pentru diferite modele de upsell ranking.

5.9 Concluzii

Rezultatele experimentale arată că modelele care folosesc informația din coș și context depășesc semnificativ metodele de tip baseline. Dintre algoritmi testati, Naive Bayes, ID3 și AdaBoost obțin cele mai bune rezultate, demonstrând că relațiile dintre produse pot fi exploatare eficient chiar și cu modele relativ simple.

De asemenea, includerea contextului temporal aduce un câștig constant de performanță, chiar dacă acesta nu este foarte mare. Acest lucru sugerează că momentul achiziției influențează comportamentul de cumpărare.

5.10 Direcții de îmbunătățire

O direcție de îmbunătățire ar putea fi includerea unor informații suplimentare disponibile în date, cum ar fi cantitatea achiziționată din fiecare produs sau frecvența cu care anumite produse apar în același coș. În acest fel, am putea diferenția mai clar produsele în funcție de relevanță.

În plus, dacă datele ar permite, am putea utiliza un istoric al comportamentului de cumpărare al clientului, care ar putea conduce la recomandări mai bine adaptate preferințelor acestuia.